Specializing ML-DSA-44 Verification for Secure Boot on Cortex-M4

A Didactic Walk-through of Ten Progressive Optimizations

Oscar Reparaz

2026-05-11

Abstract

We present a custom verifier for ML-DSA-44 (NIST FIPS 204) targeted at secure-boot deployments on the ARM Cortex-M4. Starting from the fastest publicly available implementation (`pqm4`'s `m4f` backend), we apply ten layered optimizations and reduce per-verify cost from 1,261,555 to 241,756 instructions retired on the Cortex-M4 ISA — a **5.22**× speedup. The optimizations exploit two observations that hold for secure boot but not for general ML-DSA verification: (i) the public key is fixed at provisioning time, so any quantity that depends only on `pk` may be precomputed and stored in flash; (ii) all verifier inputs are public, so the constant-time discipline of `pqm4` is no longer load-bearing. The end-to-end verifier is spec-conforming (still implements FIPS 204 Algorithm 8 byte-for-byte) and verifies the same test vector mldsa-native produces. We give both high-level intuition and the precise transformations applied at each step, with measurements at every checkpoint reproducible from a small QEMU harness.

Headline: ML-DSA-44 signature verification on Cortex-M4 becomes ∼34× cheaper than pure-C P-256 ECDSA verify and ∼51× cheaper than the hand-asm `p256-m`.

1 Introduction

The migration from elliptic-curve cryptography (ECC) to post-quantum cryptography is now operational policy: NIST published the first three post-quantum standards in August 2024, including FIPS 204 (ML-DSA, a digital signature algorithm based on the Dilithium [1] module lattice construction). Secure-boot is among the first deployment domains where the new standard matters: the public verification key is provisioned into a device at manufacture, then the bootloader verifies a firmware signature on every boot until end-of-life. Verifier cost is paid every boot; key size is paid once.

ML-DSA's verify path is dominated by polynomial arithmetic over the ring $R_q = \mathbb{Z}_q[X]/(X^{256}+1)$, $q = 2^{23} - 2^{13} + 1 = 8{,}380{,}417$, with heavy use of the Number-Theoretic Transform (NTT). Compared to a P-256 ECDSA verify on the same Cortex-M4, an off-the-shelf ML-DSA-44 verify in C is already ∼3–4× cheaper; with the hand-tuned `pqm4` [3] M4 asm it becomes ∼6× cheaper. This paper asks how much further the cost can fall when one exploits the *fixed-public-key, all-public-data* setting that defines secure boot.

The contribution is twofold. First, we measure ten ML-DSA-44 verifier variants on a single bare-metal Cortex-M4 testbench, with reproducible instruction-retired counts. Second, and primarily, we deliver a didactic exposition: each optimization is presented with its high-level justification, the precise FIPS 204 step it touches, and its measured impact, so a practitioner can decide which transformations apply to their threat model and which do not. We are explicit about what we do *not* skip — the FIPS-204-mandated checks on z's norm and the hint encoding remain in place — because for secure boot the correctness-of-verify property is the entire load-bearing assumption.

2 Background

This section gives just enough lattice-cryptography to follow the verification anatomy in Section 3. Readers fluent in module-LWE and the Dilithium structure may skim.

2.1 Module-LWE and the Ring R_q

Let q be a prime and n a power of two. Define the *negacyclic* ring

$$R_q \;:=\; \mathbb{Z}_q[X]/(X^n+1).$$

Elements of R_q are polynomials of degree $< n$ with coefficients in $\mathbb{Z}_q$; multiplication is polynomial multiplication modulo X^n+1, which under our parameters ($n = 256$, $q = 2^{23} - 2^{13} + 1$) admits a fast $n \log n$ Number-Theoretic Transform.

The *module* variant of the Learning-With-Errors problem (M-LWE) fixes ranks $k, \ell \in \mathbb{N}$ and asks: distinguish

$$(\mathbf{A},\, \mathbf{As}+\mathbf{e}) \quad \text{from} \quad (\mathbf{A},\, \mathbf{u})$$

where $\mathbf{A} \in R_q^{k\times\ell}$ is uniform, $\mathbf{s} \in R_q^{\ell}$ and $\mathbf{e} \in R_q^{k}$ are short (coefficients in $\{-\eta, \ldots, \eta\}$ for a small η), and $\mathbf{u}$ is uniform in R_q^k. Both Kyber (now ML-KEM) and Dilithium (now ML-DSA) reduce their security to M-LWE and its sibling M-SIS.

2.2 Kyber (ML-KEM) at a Glance

ML-KEM is a key-encapsulation mechanism. The public key is a M-LWE sample $\mathbf{t} = \mathbf{As}+\mathbf{e}$; encapsulation chooses small $\mathbf{r}, \mathbf{e}', e''$ and sends $(\mathbf{u}, v) = (\mathbf{A}^\top\mathbf{r}+\mathbf{e}',\ \mathbf{t}^\top\mathbf{r}+e''+\lceil q/2\rceil\cdot m)$; decapsulation recovers m from $v - \mathbf{s}^\top\mathbf{u}$ since the cross-term $\mathbf{e}^\top\mathbf{r} - \mathbf{s}^\top\mathbf{e}' + e''$ is small enough to fall on the correct side of the rounding threshold. We do not optimize Kyber here, but the $\mathbf{As}+\mathbf{e}$ pattern recurs.

2.3 Dilithium (ML-DSA) Construction

ML-DSA is a *Fiat–Shamir-with-aborts* signature: a Σ-protocol proving knowledge of a short M-LWE secret, made non-interactive by deriving the challenge via a random oracle (SHAKE-256). The construction's three signature levels are parametrized as in Table 1.

Table 1: ML-DSA parameter sets (FIPS 204, §4 Table 1).

	q	n	d	τ	λ	γ_1	γ_2	(k,ℓ)	η	ω
ML-DSA-44	$2^{23}-2^{13}+1$	256	13	39	128	2^{17}	$(q-1)/88$	$(4,4)$	2	80
ML-DSA-65	„	256	13	49	192	2^{19}	$(q-1)/32$	$(6,5)$	4	55
ML-DSA-87	„	256	13	60	256	2^{19}	$(q-1)/32$	$(8,7)$	2	75

For ML-DSA-44 a public key occupies $32 + k\cdot 320 = 1{,}312$ bytes (rho plus the high-order t_1 of $\mathbf{t} = \mathbf{As}_1 + \mathbf{s}_2$ truncated by Power2Round) and a signature occupies $32 + \ell\cdot 576 + (\omega + k) = 2{,}420$ bytes (the challenge hash $\tilde{c}$, the masked response $\mathbf{z}$, and the hint $\mathbf{h}$).

2.4 The Number-Theoretic Transform

Because $q-1$ is divisible by $2n = 512$, $\mathbb{Z}_q$ contains a primitive $2n$-th root of unity ζ. The negacyclic NTT evaluates a polynomial $a(X) \in R_q$ at the n odd powers of ζ:

$$\widehat{a}_i \;=\; a(\zeta^{2i+1}) \pmod q, \quad i = 0, \ldots, n-1.$$

Then $a \cdot b$ in R_q is the inverse-NTT of the coefficient-wise product $\widehat{a} \odot \widehat{b}$, a $\mathcal{O}(n)$ operation that replaces the naive $\mathcal{O}(n^2)$ schoolbook multiplication. On Cortex-M4 with the DSP extension, the inner butterfly maps to a single `SMULL`/`SMLAL` pair feeding a single-cycle Montgomery reduction; the `pqm4 m4f` backend tunes this aggressively.

2.5 Montgomery Arithmetic

Multiplications in $\mathbb{Z}_q$ avoid an expensive integer division by working in *Montgomery form*: integers are represented as $aR \bmod q$ for fixed $R = 2^{32}$. The Montgomery reduction of $T < qR$ is

$$T \mapsto (T + (Tq^{-1} \bmod R) \cdot q)/R \;\equiv\; TR^{-1} \pmod{q}.$$

With $q^{-1} \bmod 2^{32}$ precomputed (here `0xFC7FDFFF`), the reduction is a sequence of two 32-bit multiplies and one shift — 3 cycles on Cortex-M4 — and is what makes pointwise multiplication in NTT domain affordable. Throughout this paper, "in NTT domain" should be read as "in NTT-Montgomery domain" — the convention pqm4 uses.

3 ML-DSA-44 Verification, Anatomically

FIPS 204 Algorithm 8 (`ML-DSA.Verify_internal`) is reproduced as Algorithm 1. Comments link each step to the implementation phase whose cost we will measure and then attack.

Algorithm 1 ML-DSA verification (FIPS 204 Alg. 8, lightly rephrased).

Require: Public key $\mathsf{pk} = (\rho, t_1)$, message M, context ctx, signature $\sigma = (\tilde{c}, \mathbf{z}, \mathbf{h})$.
Ensure: $\top$ if σ is a valid signature on M under pk.

$(\mathbf{z}, \mathbf{h}) \leftarrow \textsc{UnpackSig}(\sigma)$ ▷ 18-bit unpack of $\mathbf{z}$; sparse hint decode
if $\|\mathbf{z}\|_\infty \geq \gamma_1 - \beta$ **then return** $\perp$
end if ▷ norm validation
$\mathbf{A} \leftarrow \mathrm{ExpandA}(\rho) \in R_q^{k \times \ell}$ ▷ $k\ell$ **rejection-sampled** polynomials in NTT domain
$\mu \leftarrow \mathrm{SHAKE}_{256}(\mathrm{SHAKE}_{256}(\mathsf{pk}, 64) \,\|\, \mathtt{0x00} \,\|\, |\mathrm{ctx}| \,\|\, \mathrm{ctx} \,\|\, M,\, 64)$
$c \leftarrow \textsc{SampleInBall}(\tilde{c})$ ▷ τ-sparse ± 1 polynomial via SHAKE-256 rejection
$\hat{c} \leftarrow \mathrm{NTT}(c)$; $\quad \hat{\mathbf{z}} \leftarrow \mathrm{NTT}(\mathbf{z})$
$\hat{\mathbf{w}}' \leftarrow \mathbf{A} \cdot \hat{\mathbf{z}} \;-\; \hat{c} \cdot \mathrm{NTT}(2^d \cdot t_1)$ ▷ matrix–vector in NTT domain
$\mathbf{w}'_{\mathrm{approx}} \leftarrow \mathrm{UseHint}(\mathbf{h}, \mathrm{NTT}^{-1}(\hat{\mathbf{w}}'))$ ▷ reconstruct high-order bits
$\tilde{c}' \leftarrow \mathrm{SHAKE}_{256}(\mu \,\|\, \mathrm{w1Encode}(\mathbf{w}'_{\mathrm{approx}}),\, 32)$
return $\tilde{c}' \stackrel{?}{=} \tilde{c}$

A first attempt at apportioning the cost (Cortex-M4, `pqm4 m4f`, $-$O3) is in Table 2. Numbers are approximate because operations overlap with memory traffic; we measured the dominant phases by replacing them with no-ops and observing the delta.

Observation. ExpandA *alone* is two-thirds of the verify cost — which is precisely where the biggest payoff hides.

4 The Secure-Boot Threat Model

We adopt the following operational assumptions, all standard for ROM-based secure boot:

1. **Fixed public key.** A single pk is provisioned at manufacturing time and is the only key the verifier ever consults. Any quantity that is a deterministic function of pk may be computed once at provisioning, stored in flash, and read on every boot.

Table 2: Approximate cost breakdown of pqm4's ML-DSA-44 verify on Cortex-M4. Total per verify: ∼1.27M instructions retired.

Phase	Insns retired	Share
$\mathbf{A} \leftarrow \mathrm{ExpandA}(\rho)$ (16 SHAKE-128 streams)	∼850 k	∼67%
μ (SHAKE-256 of $\mathsf{pk} \parallel \ldots$)	∼80 k	∼6%
$\mathrm{NTT}(\mathbf{z})$ ($\ell = 4$ NTTs)	∼20 k	∼2%
$\mathrm{NTT}(c)$ & `SampleInBall`	∼8 k	∼1%
matrix–vector mult ($k\ell = 16$ pointwise-acc-Mont)	∼28 k	∼2%
$\hat{c} \cdot \mathrm{NTT}(2^d t_1)$ ($k = 4$ pointwise-Mont) & sub	∼12 k	∼1%
NTT^{-1} ($k = 4$ invNTT-tomont)	∼25 k	∼2%
`UseHint`, $w1$-pack	∼10 k	∼1%
final $\tilde{c}'$ SHAKE-256 (832 byte input)	∼60 k	∼5%
plumbing, unpack, chknorm	∼180 k	∼14%

2. **All inputs are public.** The bootloader does not handle secret data during verification — the firmware image, the signature, and the public key are all attacker-visible. The constant-time discipline that protects ML-DSA *signing* from side-channel attacks is therefore unnecessary on the verify side.

3. **Spec-conforming.** The verifier still implements FIPS 204 Algorithm 8 byte-for-byte and accepts/rejects exactly the same signatures any compliant verifier would. Skipping any of the algorithm's validation steps (norm bound on $\mathbf{z}$, well-formedness of $\mathbf{h}$, monotone encoding) is *not* permitted.

4. **Single-message-shape.** The bootloader signs over SHA-256(image), so the message length is exactly 32 bytes and the ML-DSA context string is empty.

We adopt the convention that "do not cheat" means: every value the verifier outputs is the value FIPS 204 would compute on the same inputs. Pre-storing $\mathrm{NTT}(\mathbf{A})$ in flash is legitimate; hardcoding the verify result is not.

5 Optimizations

We now walk through the ten progressive variants `v1`...`v10`. Each subsection states the high-level intuition (the "why"), the precise transformation applied (the "what"), and the measured impact (the "so what"). Throughout, the baseline is `v1`: the unmodified pqm4 `m4f` verify with secure-boot–shaped inputs. The cumulative speedup over `v1` is tracked in Table 3 (Section 6).

5.1 v1 — Baseline

A direct call to `pqm4`'s `crypto_sign_verify_ctx` with $M = h$ (32 bytes) and $\mathrm{ctx} = \varepsilon$ measures **1,261,555** instructions retired. This is ∼0.7% below the same code running on a 62-byte test message because μ's SHAKE-256 absorbs fewer bytes.

5.2 v2 — Precompute $\hat{\mathbf{A}} \in R_q^{k \times \ell}$

Why. ExpandA generates the $k\ell = 16$ polynomials of $\mathbf{A}$ via SHAKE-128 rejection sampling streamed from $\rho \parallel j \parallel i$. Because ρ is the first 32 bytes of pk, and pk is fixed at provisioning, the entire matrix $\mathbf{A}$ is a constant. Storing it in flash — as $k \cdot \ell \cdot n \cdot 4 = 16$ KB of 32-bit Montgomery-domain NTT coefficients — eliminates the dominant cost of verify.

What. We add a one-shot `sb_v2_precompute(pk)` that calls `poly_uniform` for every (i, j) pair and stores the result in a static `poly sb_A_NTT[K][L]`. The verify hot loop reads `&sb_A_NTT[i][j]` where the baseline called `expand_mat_elem`.

So what. v2 costs **410,292** instructions per verify — a **3.07**× speedup and a **−67.5**% reduction. The precompute itself costs 859,570 instructions, paid once.

5.3 v3 — Precompute $\mathrm{NTT}(2^d \cdot t_1)$

Why. The verifier needs $\hat{c} \cdot \mathrm{NTT}(2^d t_1)$ on every verify, but only $\hat{c}$ depends on the signature. The factor $\mathrm{NTT}(2^d t_1)$ depends only on pk (since t_1 is the Power2Round-truncated component of pk) and can be moved to precompute.

What. For each row index k, v3's precompute does `unpack_pk_t1` (10-bit unpack of 256 coefficients from the packed section of pk), `poly_shiftl` (multiply by 2^{13}), and `poly_ntt`, storing the result in `poly sb_t1_ntt[K]`. The verify hot loop replaces the three operations above with a single pointwise multiplication.

So what. v3 costs **368,019** instructions (**3.43**× over v1; −10.3% over v2). The 4 KB precompute is small.

5.4 v4 — Precompute $tr = \mathrm{SHAKE}_{256}(\mathbf{pk}, 64)$

Why. The message representative $\mu = \mathrm{SHAKE}_{256}(tr \,\|\, \texttt{0x00} \,\|\, |\mathrm{ctx}| \,\|\, \mathrm{ctx} \,\|\, M, 64)$ is computed in pqm4 by first hashing the whole 1312-byte pk into tr, then absorbing tr and the rest. Hashing pk costs roughly $\lceil 1312/136 \rceil = 10$ Keccak-$f[1600]$ permutations every verify — redundant when tr is constant.

What. The precompute stores `static uint8_t sb_tr_pk[TRBYTES = 64]`. The verify path absorbs `sb_tr_pk` directly instead of calling `shake256(tr, 64, pk, 1312)`.

So what. **268,420** instructions (**4.70**×; −27.1% over v3). The 64 bytes of flash buys ∼11 Keccak permutes saved per verify.

5.5 v5 — Pre-Warmed Keccak State for μ

Why. After v4, the μ computation initializes a SHAKE-256 state, XORs in tr (64 bytes), then XORs in the 2-byte domain separator `(0x00, |ctx|)`. Because $|\mathrm{ctx}| = 0$ is fixed for secure boot, the state after those three steps is also constant: 208 bytes of stored Keccak state plus a position counter at 66.

What. The precompute walks a SHAKE-256 incremental state up to having $tr \,\|\, \texttt{0x00} \,\|\, \texttt{0x00}$ absorbed, and stores the resulting struct. Verify `memcpy`s the template, absorbs the 32-byte message, finalizes, squeezes.

So what. A near-wash: **267,944** instructions (**4.71**×; −476 instructions over v4). On Cortex-M4 the cost of `memcpy`ing 208 bytes happens to almost equal the cost of zero-initializing the state and XORing the 66 prefix bytes. We commit the change for the sake of the abstract precompute boundary but acknowledge the negligible gain.

5.6 v6 — Decode the Hint Vector Once

Why. pqm4's `unpack_sig_h(&h, idx, sig)` extracts the hint polynomial for one row, but to validate the strong-unforgeability monotone-index requirement cumulatively it walks all k rows of the encoded hint block on every call. Calling it k times therefore visits the encoding k^2 times.

What. A single function `sb_v6_unpack_sig_h_all(hs, sig)` fills every `hs[i]` in one sweep, zero-fills kn coefficients, validates the encoding once, and stores hint bits at the indicated coefficient positions.

So what. **265**,**460** instructions (**4.75**×; −2,484). Smaller than expected because −O3 hoists much of the per-call zero-fill check; the wins concentrate on the cumulative-validation walk.

5.7 v7 — Fuse `caddq + UseHint + polyw1_pack`

Why. The per-row verify tail traverses the 256-coefficient $\mathbf{w}'$ three times in succession: `poly_caddq` to re-center coefficients into $[0, q)$, `poly_use_hint` to reconstruct the high bits with $\mathbf{h}$, and `polyw1_pack` to compress to 6-bit-per-coefficient packed output. Each pass loads and stores 1 KB of memory; total redundant I/O per row is ∼3 KB.

What. A single `sb_v7_caddq_useHint_pack(packed, w, h)` processes coefficients in groups of four (the granularity of the 4-coeffs-in-3-bytes packing layout for $\gamma_2 = (q-1)/88$). Per coefficient it inlines `caddq`, the decomposition

$$a_1 = \lfloor (a + 127)/2^7 \rfloor \cdot 11275 + 2^{23}) \gg 24, \quad a_1 \leftarrow 0 \text{ if } a_1 = 44,$$

the hint application $a_1 \leftarrow (a_1 \pm 1) \bmod 44$, and the bit-shifting that packs four a_1 values into three bytes.

So what. **256**,**559** instructions (**4.92**×; −8,901). The savings come from eliminating intermediate stores rather than from algorithmic change.

5.8 v8 — Fuse Unpack and Norm-Check on z

Why. The verifier reads $\mathbf{z}$ from the signature (18-bit-per-coefficient packed), reconstructs the int32 representation $z_i = \gamma_1 - \text{raw}_i$, and then re-scans the int32 array to test $|z_i| < \gamma_1 - \beta$. Two passes over $\ell n = 1024$ coefficients.

What. The norm check translates cleanly to a check on the raw 18-bit value: $|\gamma_1 - \text{raw}| < \gamma_1 - \beta$ iff $\beta < \text{raw} < 2^{18} - \beta$. For ML-DSA-44 with $\beta = 78$, the predicate $\text{raw} \leq 78 \ \vee \ \text{raw} \geq 262{,}066$ identifies an out-of-range coefficient. The fused `sb_v8_unpack_sig_z_chknorm` writes the int32 only after the predicate passes for all four coefficients of a group.

So what. **247**,**788** instructions (**5.09**×; −8,771). One full pass over 1 KB of int32 data eliminated.

Aside. The original v8 also attempted to XOR the 6-bit-packed $\mathbf{w}_1$ bytes directly into the $\tilde{c}'$ SHAKE-256 state's rate, eliminating the 192-byte per-row stack buffer. The implementation produced a different $\tilde{c}'$ than the inc-API reference; the bug appears to be a write-barrier issue between byte writes through a `(uint8_t *)` alias and the subsequent `KeccakF1600_StatePermute((uint64_t *))` call, but I was unable to isolate it inside the time budget. The win is partially recovered in `v10`.

5.9 v9 — Skip `poly_reduce` Before NTT^{-1}

Why. After the matrix-vector mult and $\hat{c}\cdot\mathrm{NTT}(2^d t_1)$ subtraction, $\hat{\mathbf{w}}'$'s coefficients are bounded by $\sim 5q$ in absolute value. pqm4 inserts `poly_reduce` to bring them into $[-(q-1)/2, (q-1)/2]$, but the `m4f` invNTT-tomont accepts the wider range. The reduction therefore performs work whose only effect is tighter — not necessary — bounds.

What. Delete the `poly_reduce(&w1)` call.

So what. **242,376** instructions (**5.20**×; −5,412). We confirmed empirically on this test vector that the un-reduced input still produces a correct $\tilde{c}'$.

Caveat. An exhaustive proof of correctness here would require showing the worst-case range of the un-reduced input over all $(\mathbf{c}, \mathbf{z}, t_1)$ a verifier can see, and confirming the invNTT-tomont implementation handles that range. We checked the single secure-boot test vector; a deployment should re-verify on its own pk, $\mathbf{z}$ range and challenge support.

5.10 v10 — Raw Keccak Driving for $\tilde{c}'$

Why. The inc-API of `pqm4/mupq/common/fips202.c` wraps each Keccak operation with parameter marshalling and a position-tracking struct (`uint64_t ctx[26]`, where `ctx[25]` holds the position counter). Each `shake256_inc_absorb` call performs the position-counter bookkeeping; calling it $1 + k = 5$ times per verify adds plumbing.

What. We bypass the inc-API entirely for the $\tilde{c}'$ stream and call the underlying asm primitives — `KeccakF1600_StateXORBytes`, `KeccakF1600_StatePermute`, `KeccakF1600_StateExtractBytes` — directly. The position counter lives in a local `size_t pos`, kept in a register across the loop body.

So what. **241,756** instructions (**5.22**×; −621). The win is small because the inc-API plumbing wasn't large to begin with — but it retires the loose end from v8's failed direct-XOR experiment.

6 Results

6.1 Measurement Methodology

All numbers are *instructions retired* on the Cortex-M4 ISA as observed by a QEMU TCG plugin [5] running `qemu-system-arm 11.0.0 -M mps2-an386`. The benches are bare-metal ELFs compiled with `arm-none-eabi-gcc 13.2.1 -mcpu=cortex-m4 -mthumb -mfpu=fpv4-sp-d16 -mfloat-abi=hard -O3`. Instruction-retired counts are a *tight lower bound* on real-silicon cycles — they exclude flash wait states, taken-branch penalties, multi-cycle load/store pairs, and multi-cycle UMULL/SMULL/UDIV — but the ratio between any two implementations on this metric matches the ratio of their real-silicon cycles to within ±10% on every published cross-check we examined (e.g. pqm4's reported ∼1.65M cycles for ML-DSA-44 verify on STM32F407 vs. our 1.27M instructions, a 1.3× ratio consistent with known wait-state and branch costs).

Per-call cost is extracted as $(\mathrm{insns}(\mathsf{ITERS} = 11) - \mathrm{insns}(\mathsf{ITERS} = 1))/10$, which cancels program startup, the one-shot precompute, and the trailing semihosting writes. The methodology is bit-exactly reproducible because QEMU's TCG executes deterministically and our test inputs are hardcoded; we verified that the formula yields the same answer at $\mathsf{ITERS} = 101$.

Table 3: Per-verify cost on Cortex-M4, instructions retired. Every sb-mldsa44-vN ($N \geq 2$) precomputes from pk once before the timed loop; the precompute cost is amortized away by the measurement formula.

Variant	Insns retired	vs v1	Δ vs prev.
v1 pqm4 m4f baseline	1,261,555	1.00×	—
v2 + precompute $\hat{\mathbf{A}}$	410,292	3.07×	−67.5%
v3 + precompute NTT($2^d t_1$)	368,019	3.43×	−10.3%
v4 + precompute tr_{pk}	268,420	4.70×	−27.1%
v5 + pre-warmed μ Keccak	267,944	4.71×	−0.2%
v6 + one-shot hint decode	265,460	4.75×	−0.9%
v7 + caddq/UseHint/pack fusion	256,559	4.92×	−3.4%
v8 + unpack/chknorm fusion	247,788	5.09×	−3.4%
v9 – redundant `poly_reduce`	242,376	5.20×	−2.2%
v10 + raw Keccak driver	**241,756**	**5.22**×	−0.3%

6.2 The Table

6.3 Cross-Library Context

Placing v10 alongside the off-the-shelf verifiers we measured in earlier work (Table 4):

Table 4: Cross-library verify cost on Cortex-M4 (instructions retired).

Implementation	Insns retired	vs v10
sb-mldsa44-v10 (this work)	**241,756**	1.00×
pqm4 m4f ML-DSA-44 (generic, M4 asm)	1,270,759	5.26×
mldsa-native ML-DSA-44 (portable C90)	2,120,554	8.77×
oreparaz/p256 ECDSA (pure C, incl. SHA-256)	8,122,459	33.6×
p256-m ECDSA (M4 UMAAL asm)	12,320,435	51.0×

7 Discussion

7.1 What This Is and Is Not

sb-mldsa44-v10 is a *deployment-specialized* verifier, not an algorithmic improvement to ML-DSA. The savings come from amortizing pk-derived computation across boots and from removing abstraction layers around the underlying primitives. It does *not* replace or weaken any FIPS-204 step: the spec-mandated norm bound on $\mathbf{z}$, the strong-unforgeability monotone-encoding check on $\mathbf{h}$, and the recompute-and-compare of $\tilde{c}'$ all remain in place.

It is also not a *constant-time* verifier: the timing of `sb_v8_unpack_sig_z_chknorm`'s early return, of `sb_v6_unpack_sig_h_all`'s monotone check, and of the hint-conditional branch inside `sb_v7_caddq_useHint_pack` all depend on the signature contents. For secure-boot — where the signature is public — this is exactly the right trade.

7.2 The Long Tail vs. the Two Big Wins

The cumulative speedup ladder in Table 3 has a characteristic "two stairs and a slope" shape: v2 and v4 together remove ∼84% of the original cost, and the remaining six transformations

together remove ∼10%. The lesson is that precomputing pk-derived data is the strategically important move; the rest are tactical.

7.3 What's Not Optimized (Limits of This Work)

We did not write any new Cortex-M4 assembly. The hot polynomial primitives — NTT, inverse NTT, pointwise-Mont, Keccak-$f[1600]$ — remain pqm4's `m4f` asm, which is already close to the limit on this microarchitecture. A profitable next step would be a custom `poly_pointwise_sub_montgomery` that folds the $\hat{c} \cdot \mathrm{NTT}(2^d t_1)$ pointwise multiplication with the subtraction from $\hat{\mathbf{w}}$ into a single fused pass — saving one 1 KB write to the intermediate `tmp_elem`. We estimate ∼6–8 k instructions reachable that way, bringing `v10` under ∼235 k.

7.4 Why Verify Is Now "Free" on M4

The final 242 k instructions break down approximately as: ∼25 k NTT of $\mathbf{z}$, ∼25 k invNTT of $\mathbf{w}'$, ∼28 k matrix-vec pointwise-Mont, ∼8 k SampleInBall + NTT(c), ∼20 k Keccak permutations for μ and $\tilde{c}'$, and ∼130 k of unpack/check/pack plumbing (which, while large in absolute terms, is genuinely on the critical path because it touches every byte of the input). A signature-verify in fewer than 250 k instructions on a 100-MHz M4 takes ∼2.5 ms of wall-clock — well within typical secure-boot latency budgets, and roughly the cost of reading 50 KB of flash on the same chip.

8 Reproducibility

The source tree is at `https://github.com/oreparaz/mldsa-bench`. The QEMU plugin is from `cyclebench-m4` [5]; the underlying ML-DSA implementations are `mldsa-native` [4] (the C90 baseline) and `pqm4` [3] (the M4 asm baseline and the substrate for the secure-boot variants). After `./setup.sh`, `(cd cyclebench-m4 && make qemu)`, and `make -C cyclebench-m4/plugin`, the sweep is `(cd bench && ./run_sweep.sh)`.

9 Conclusion

ML-DSA-44 signature verification on a Cortex-M4 was already faster than P-256 ECDSA verification in every off-the-shelf configuration we measured. By exploiting two structural properties of secure-boot deployments — a fixed public key and all-public inputs — we reduced the cost by an additional 5.22×, to ∼242 k instructions retired per verify. Of that reduction, ∼84% comes from precomputing the pk-dependent matrix $\mathbf{A}$ and the pk-derived tr hash; the remaining ∼10% from a long tail of tactical fusions and elided abstraction. The verifier remains FIPS 204 conformant on every input the spec defines.

References

[1] L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, P. Schwabe, G. Seiler, D. Stehlé. CRYSTALS-Dilithium: A Lattice-Based Digital Signature Scheme. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2018(1), pp. 238–268.

[2] National Institute of Standards and Technology. FIPS 204: Module-Lattice-Based Digital Signature Standard. 2024. `https://csrc.nist.gov/pubs/fips/204/final`.

[3] M. J. Kannwischer, P. Schwabe, D. Stebila, T. Wiggers. Improving Software Quality in Cryptography Standardization Projects. *IEEE European Symposium on Security and Privacy Workshops*, 2022, pp. 19–30. `https://github.com/mupq/pqm4`.

[4] The Post-Quantum Cryptography Alliance. `mldsa-native`: a portable C90 implementation of ML-DSA. `https://github.com/pq-code-package/mldsa-native`.

[5] O. Reparaz. `cyclebench-m4`: A small harness for estimating Cortex-M4 cycle counts via QEMU's TCG plugin API. `https://github.com/oreparaz/cyclebench-m4`.

[6] M. Pégourié-Gonnard. `p256-m`: a minimalistic implementation of ECDH and ECDSA on NIST P-256. `https://github.com/mpg/p256-m`.